

# UNIVERSIDAD NACIONAL DE INGENIERÍA

## FACULTAD DE CIENCIAS

ESCUELA PROFESIONAL DE CIENCIAS DE LA COMPUTACIÓN



## INTELIGENCIA ARTIFICIAL – CC421A

**Trabajo : CNN - Algoritmo YOLO**

### **Integrantes:**

- Vilca Perez, Mathias Joaquin - 20234070J
- Urbano Nieves, Ivan Marcelo - 20220461A
- Pumayauli Evangelista, Bryan José - 20212209E
- Santa Cruz Basilio, Jesus Diego - 20235005G
- López Arteaga, César Omar - 20236743A

**Profesor: Marcos Antonio Alania Vicente**

# Contenido

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduccion</b>                                       | <b>2</b>  |
| 1.1      | Objetivos . . . . .                                       | 2         |
| <b>2</b> | <b>Red Neuronal Convolutacional(CNN)</b>                  | <b>3</b>  |
| 2.1      | Componentes de una Red Neuronal Convolutacional . . . . . | 3         |
| 2.1.1    | Capas convolucionales . . . . .                           | 3         |
| 2.2      | Agrupación de Capas(Pooling) . . . . .                    | 8         |
| 2.2.1    | Capas convolucionales adicional . . . . .                 | 9         |
| 2.2.2    | Capa completamente conectada . . . . .                    | 10        |
| <b>3</b> | <b>Algoritmo YOLO</b>                                     | <b>11</b> |
| 3.1      | Arquitectura . . . . .                                    | 11        |
| 3.2      | Deteccion de Objetos . . . . .                            | 12        |
| 3.2.1    | Bloques residuales . . . . .                              | 13        |
| 3.2.2    | Regresion de Caja Delimitadora . . . . .                  | 13        |
| 3.2.3    | Intersección sobre Union (IOU) . . . . .                  | 15        |
| 3.2.4    | Non maximum suppression (NMS) . . . . .                   | 17        |
| 3.3      | Desarrollo de algoritmo . . . . .                         | 17        |
| <b>4</b> | <b>Implementación de YOLO11</b>                           | <b>19</b> |
| 4.1      | Modelos preentrenados . . . . .                           | 20        |
| 4.2      | Preparación de dataset . . . . .                          | 21        |
| 4.3      | Modelos corridos . . . . .                                | 23        |
| 4.4      | Resultados de métricas . . . . .                          | 26        |
| <b>5</b> | <b>Conclusiones</b>                                       | <b>29</b> |
| <b>6</b> | <b>Bibliografia</b>                                       | <b>30</b> |

# 1 Introduccion

Las Redes Neuronales Espectrales son un tipo de redes neuronales que operan en el dominio de la frecuencia en lugar del dominio espacial o temporal. Se basan en herramientas matemáticas como la transformada de Fourier, transformada de Laplace y la transformada de Ondas (Wavelets) para extraer características, acelerar cálculos y mejorar la generalización.

Dentro de la Redes Neuronales Espectrales, en funcion a lo que se busca, se puede clasificar en :

- Redes Neuronales Fourier (Fourier Neural Networks - FNN)
- Redes Espectrales en Grafos (Graph Spectral Networks - GNN Espectrales)
- Operadores Neuronales Fourier (Fourier Neural Operators - FNO)
- Redes Neuronales de Ondas (Wavelet Neural Networks - WNN)

## 1.1 Objetivos

En el presente trabajo, se estudiara las FNN, para lo cual se tendra los siguientes objetivos:

- Revision de la teoria fundamental de la series de Fourier, trasnformada de Fourier y del algortimo FFT
- Aplicacion de la transformada de Fourier en aplicacion de imagenes.
- Revision de Redes Neuronales Convolucionales.
- Estudio del algoritmo YOLO.

En el capitulo 2 se hace estudio generel para poder ser utilizado en todo el trabajo. En el capitulo 3 se hace aplicacion del analisis de fourier en procesamiento de imagenes. capitulo 4 se hace la revision de las redes neuronales Convolucionales. En el capitulo 5 se hace el estudio de las FNN. El capitulo 6 se hace el estudio del algoritmo YOLO.

## 2 Red Neuronal Convolucional(CNN)

Una red neuronal convolucional (CNN), es un tipo especializado de algoritmo de aprendizaje profundo diseñado principalmente para tareas que requieren reconocimiento de objetos. Este tipo de red es una variación de un perceptron multicapa, sin embargo, debido a que su aplicación es realizada en matrices bidimensionales, son muy efectivas para tareas de visión artificial, como en la clasificación y segmentación de imágenes, entre otras aplicaciones.

Asi la definición matemática de una Red Neuronal Convolucional (CNN), se puede representarse como una composición de funciones de la forma:

$$\mathcal{F}(x) = f_L \circ f_{L-1} \circ \dots \circ f_1(x)$$

donde cada  $f_\ell$  representa una capa de la red y puede ser una convolución, una activación no lineal, o una operación de *pooling*.

### 2.1 Componentes de una Red Neuronal Convolucional

La red neuronal convolucional se compone de cuatro partes principales. Estas componentes ayudan a las CNN a imitar el funcionamiento del cerebro humano para reconocer patrones y características en las imágenes:

- Capas convolucionales
- Agrupamiento de Capas
- Capas convolucionales adicional
- Capas totalmente conectadas

#### 2.1.1 Capas convolucionales

La capa de convolución es el elemento fundamental de la CNN. Lleva la mayor parte de la carga computacional de la red. Asi esta capa de convolucion, requiere algunos componentes, que son datos de entrada, un filtro(kernel) y un mapa de características.

Supongamos que la entrada será una imagen en color, que se compone de una matriz de píxeles en 3D. Esto significa que la entrada tendrá tres dimensiones (altura, ancho y profundidad) que corresponden a RGB en una imagen.

El detector de características, conocido como kernel o filtro, que se moverá a través de los campos receptivos de la imagen (matrices de datos de la imagen), verificando si la característica está presente, eso se realiza haciendo el producto escalar de píxeles entre el kernel y el campo receptivo de la imagen. Este proceso se conoce como convolución. El núcleo es espacialmente más pequeño que una imagen, pero es más profundo. Esto significa que, si la imagen está compuesta de tres canales (RGB), la altura y el ancho del núcleo serán espacialmente pequeños, pero la profundidad se extenderá hasta los tres canales.

El detector de características es una matriz bidimensional (2-D) de pesos, que representa parte de la imagen. Aunque esto puede variar, el tamaño del filtro suele ser una matriz  $3 \times 3$ ; esto también determina el tamaño del campo receptivo. Luego de la aplicación de la convolución se introduce entonces en una matriz de salida. Después, el filtro cambia a pasos agigantados, repitiendo el proceso hasta que el kernel ha barrido toda la imagen. El resultado final de la serie de productos de puntos de la entrada y el filtro se conoce como mapa de características, mapa de activación o característica convolucionada.

Los pesos en el detector de características permanecen fijos a medida que se mueve por la imagen, lo que también se conoce como intercambio de parámetros. Algunos parámetros, como los valores de peso, se ajustan durante el entrenamiento a través del proceso de retropropagación y descenso de gradiente. Sin embargo, hay tres hiperparámetros que afectan el tamaño del volumen de la salida y que deben establecerse antes de que comience el entrenamiento de la red neuronal. Estos incluyen:

1. El número de filtros afecta la profundidad de la salida. Por ejemplo, tres filtros distintos producirían tres mapas de características diferentes, creando una profundidad de tres.

2. Stride es la distancia, o número de píxeles, que el núcleo se desplaza sobre la matriz de entrada. Aunque los valores de stride iguales o superiores a dos son poco frecuentes, un stride mayor produce una salida menor.
3. El zero-padding se usa generalmente cuando los filtros no se ajustan a la imagen de entrada. De este modo, todos los elementos que quedan fuera de la matriz de entrada se ponen en cero, lo que produce una salida mayor o del mismo tamaño. Hay tres tipos de relleno:
  - Relleno válido: Esto también se conoce como "sin relleno". En este caso, la última convolución se descarta si las dimensiones no se alinean.
  - Mismo relleno: este relleno garantiza que la capa de salida tenga el mismo tamaño que la capa de entrada.
  - Relleno completo: este tipo de relleno aumenta el tamaño de la salida al agregar ceros al borde de la entrada.

Así la operación de convolución discreta bidimensional se puede representar matemáticamente como:

Dada una entrada  $x \in \mathbb{R}^{H \times W}$  y un *filtro*  $K \in \mathbb{R}^{k \times k}$ , la operación de convolución discreta 2D (con padding y stride) está definida como:

$$(x * K)(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} x(i \cdot s + m - p, j \cdot s + n - p) \cdot K(m, n)$$

Donde:

- $x(i, j)$ : valor de la entrada (por ejemplo, imagen) en la posición  $(i, j)$ .
- $K(m, n)$ : valor del filtro (o kernel) en la posición  $(m, n)$ , de tamaño  $k \times k$ .
- $s$ : **stride** (paso) de la convolución: cuánto se desplaza el filtro en cada paso horizontal o vertical.
- $p$ : **padding**: número de ceros añadidos alrededor del borde de la entrada para controlar el tamaño de la salida.
- $(f * x)(i, j)$ : valor de la salida (feature map) en la posición  $(i, j)$ .

### Observaciones:

- Si  $p = 0$ , la convolución es **válida** y la salida es más pequeña que la entrada.
- Si  $p$  se elige de modo que preserve el tamaño, se habla de **convolución con mismo tamaño** (*same convolution*).
- Un valor  $s > 1$  implica submuestreo, pues el filtro salta posiciones.

La salida resultante de esta operación es un nuevo mapa que realiza ciertas características locales dependiendo del filtro utilizado. En una red CNN, se usan múltiples filtros, cada uno aprendiendo una representación distinta.

Después de cada operación de convolución, una CNN aplica una transformación de Unidad lineal rectificada (ReLU) al mapa de características, introduciendo la no linealidad en el modelo.

$$\text{ReLU}(x) = \max(0, x)$$

Se prefiere usar ReLU sobre otras funciones de activación clásicas (como sigmoide o  $\tanh$ ) por varias razones matemáticas y computacionales fundamentales. Primero que evita el problema del gradiente desvanecido, es decir, las funciones clásicas como:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (\text{sigmoide}), \quad \text{y} \quad \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

tienen derivadas que tienden a cero cuando  $x \rightarrow \pm\infty$ , lo cual provoca que los gradientes durante el entrenamiento se vuelvan muy pequeños. Esto genera aprendizaje lento o estancamiento. En cambio, la función ReLU:

$$\text{ReLU}(x) = \max(0, x), \quad \text{entonces} \quad \text{ReLU}'(x) = \begin{cases} 1 & \text{si } x > 0 \\ 0 & \text{si } x \leq 0 \end{cases}$$

mantiene el gradiente activo en el dominio positivo.

Lo segundo por que se usa ReLU es porque la CNN requieren no linealidad para aprender funciones complejas. A diferencia de la sigmoide y la tangente hiperbólica,

que saturan en valores extremos (aplastamiento), ReLU no satura para  $x > 0$ , permitiendo un aprendizaje más efectivo. La tercera razón por la que usa el ReLU es por la eficiencia computacional, es decir que la operación  $\max(0, x)$  es simple de implementar computacionalmente, sin necesidad de exponenciales ni divisiones. Esto acelera tanto el entrenamiento como la inferencia.

A continuación se muestra un ejemplo de aplicación de convolución (Fig. 1), así durante el paso hacia adelante, el núcleo se desliza a lo largo de la altura y el ancho de la imagen, produciendo la representación de la imagen de esa región receptiva. Esto produce una representación bidimensional de la imagen conocida como mapa de activación que proporciona la respuesta del núcleo en cada posición espacial de la imagen. El tamaño deslizante del núcleo se llama zancada (Stride)

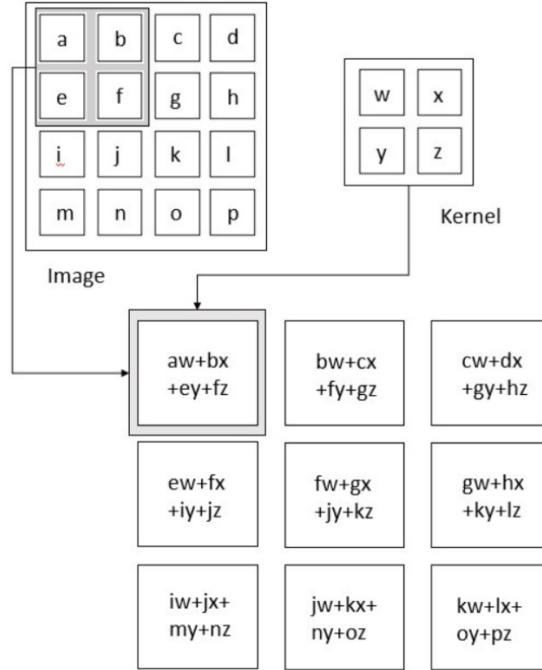


Figure 1: Se muestra la convolucion, llegando a al mapa de activacion

Si tenemos una entrada de tamaño  $W \times W \times D$  y un número de núcleos  $D_{out}$  con un tamaño espacial de  $F$  con paso  $S$  y cantidad de relleno  $P$ , entonces el tamaño del volumen de salida se puede determinar con la siguiente fórmula:

$$W_{out} = \frac{W - F + 2P}{S} + 1 \quad (1)$$



Esto producirá un volumen de salida de tamaño  $W_{out} \times W_{out} \times D_{out}$ .

## 2.2 Agrupación de Capas(Pooling)

La agrupación de capas, también conocida como reducción de muestreo, reduce la dimensionalidad y, por lo tanto, el número de parámetros en la entrada. De manera similar a la capa convolucional, la operación de agrupación muestra un filtro en toda la entrada, pero la diferencia es que este filtro no tiene pesos. En cambio, el kernel aplica una función de agregación a los valores dentro del campo receptivo, poblando la matriz de salida. Hay dos tipos principales de agrupación:

- Agrupación máxima(Maxpooling): a medida que el filtro se mueve a través de la entrada, selecciona el píxel con el valor máximo para enviar a la matriz de salida. Como dato informativo, este enfoque tiende a usarse con más frecuencia en comparación con la agrupación promedio.

$$\text{MaxPooling}(x) = \max\{x_{ij} \mid i, j \in \text{ventana}\}$$

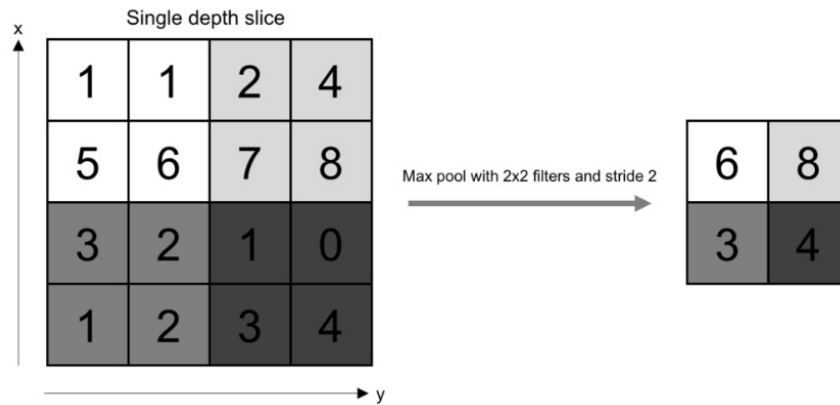


Figure 2: Capa de agrupamiento - Max pooling

- Agrupación promedio(AveragePooling): a medida que el filtro se mueve a través de la entrada, calcula el valor promedio dentro del campo receptivo para enviarlo a la matriz de salida.

Si tenemos un mapa de activación de tamaño  $W \times W \times D$ , un núcleo de agrupamiento de tamaño espacial  $F$  y un paso  $S$ , entonces el tamaño del volumen de salida se puede

determinar mediante la siguiente fórmula:

$$W_{out} = \frac{W - F}{S} + 1 \quad (2)$$

Esto producirá un volumen de salida de tamaño  $W_{out} \times W_{out} \times D$ .

En todos los casos, la agrupación proporciona cierta invariancia de traducción, lo que significa que un objeto sería reconocible independientemente de dónde aparezca en el cuadro.

### 2.2.1 Capas convolucionales adicionales

Otra capa de convolución puede seguir a la capa de convolución inicial. Cuando esto sucede, la estructura de la CNN puede convertirse en jerárquica, ya que las capas posteriores pueden ver los píxeles dentro de los campos receptivos de las capas anteriores. Por ejemplo, supongamos que intentamos determinar si una imagen contiene una bicicleta. Pensemos en la bicicleta como una suma de partes. Se compone de un cuadro, manillar, ruedas, pedales, etc. Cada parte individual de la bicicleta constituye un patrón de nivel inferior en la red neuronal, y la combinación de sus piezas representa un patrón de nivel superior, creando una jerarquía de características dentro de la CNN. En última instancia, la capa convolucional convierte la imagen en valores numéricos, lo que permite a la red neuronal interpretar y extraer patrones relevantes.



Figure 3: Vista de capas adicionales convolucionales

### 2.2.2 Capa completamente conectada

El nombre de la capa totalmente conectada alude a que a una red neuronal full connection (FC). Como se mencionó antes, los valores de píxeles de la imagen de entrada no se conectan de manera directa a la capa de salida en capas parcialmente conectadas. Sin embargo, en la capa totalmente conectada, cada nodo de la capa de salida establece una conexión directa a un nodo en la capa anterior.

Esta capa realiza la tarea de clasificación basada en las características extraídas a través de las capas anteriores y sus diferentes filtros. Mientras que las capas convolucionales y agrupadas tienden a usar funciones ReLu, las capas FC suelen aprovechar una función de activación softmax para clasificar las entradas adecuadamente, produciendo una probabilidad de 0 a 1.

## 3 Algoritmo YOLO

You Only Look Once (YOLO) es un algoritmo de detección de objetos en tiempo real de última generación introducido en 2015 por Joseph Redmon, Santosh Divvala, Ross Girshick y Ali Farhadi en su famoso trabajo de investigación "You Only Look Once: Detección de Objetos Unificada y en Tiempo Real".

Los autores enmarcan el problema de la detección de objetos como un problema de regresión en lugar de una tarea de clasificación, separando espacialmente las cajas delimitadoras y asociando probabilidades a cada una de las imágenes detectadas mediante una única red neuronal convolucional (CNN).

### 3.1 Arquitectura

El modelo se implementó como una red neuronal convolucional y fue evaluado en el set de datos para detección de PASCAL VOC. Las capas convolucionales iniciales de la red se encargan de la extracción de características de la imagen, mientras que las capas de conexión completa predicen la probabilidad de salida y las coordenadas del objeto.

La red tiene 24 capas convolucionales seguidas por 2 capas de conexión completa; esta hace uso de capas de reducción de 1x1 seguidas capas convolucionales de 3x3. El modelo Fast YOLO hace uso de una red neuronal de 9 capas. La salida final del modelo tensor de predicción de 7x7x30.[3]

La arquitectura de YOLO, tiene en total 24 capas convolucionales, cuatro capas de agrupamiento máximo y dos capas totalmente conectadas.

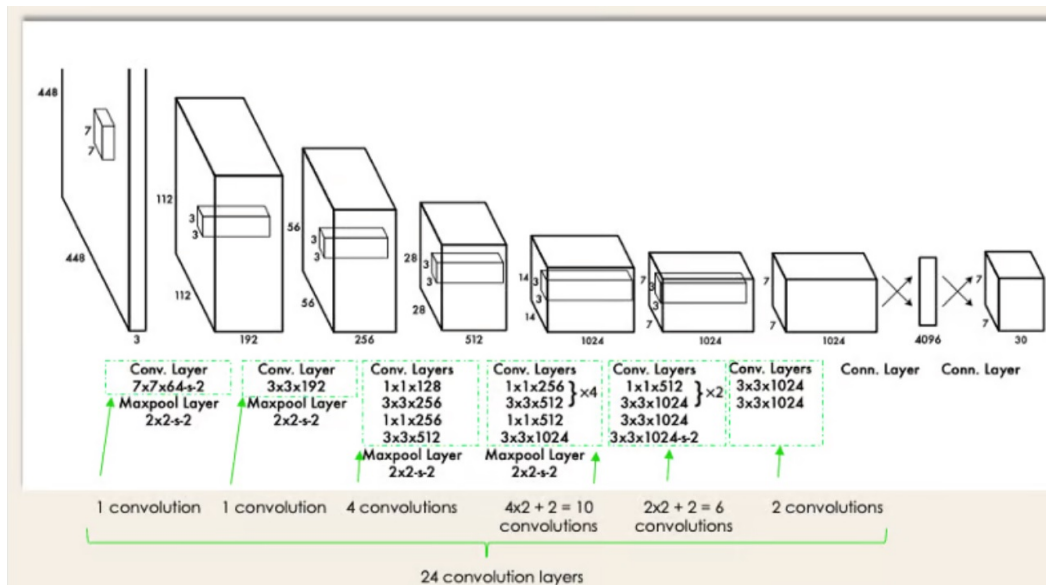


Figure 4: Descripción de la imagen

La arquitectura funciona como sigue:

- Cambia el tamaño de la imagen de entrada a 448x448 antes de pasar por la red convolucional.
- Primero se aplica una convolución 1x1 para reducir el número de canales, a la que sigue una convolución 3x3 para generar una salida cuboidal.
- La función de activación bajo el capó es ReLU, excepto en la capa final, que utiliza una función de activación lineal.
- Algunas técnicas adicionales, como la normalización por lotes y el abandono, regularizan respectivamente el modelo y evitan que se ajuste en exceso.

## 3.2 Deteccion de Objetos

El algoritmo funciona basándose en los cuatro enfoques siguientes:

- Bloques residuales.
- Regresión de caja delimitadora.
- Intersección sobre la union(IOUS).
- Non maximum suppression.

### 3.2.1 Bloques residuales

Este primer paso comienza dividiendo la imagen original (A) en celdas de cuadrícula  $N \times N$  de igual forma. Cada celda de la cuadrícula es responsable de localizar y predecir la clase del objeto que abarca, junto con el valor de probabilidad/confianza. En la figura 5, en el lado derecho se ve la imagen original, en el lado izquierdo, se observa la imagen con la división para  $N=4$ .

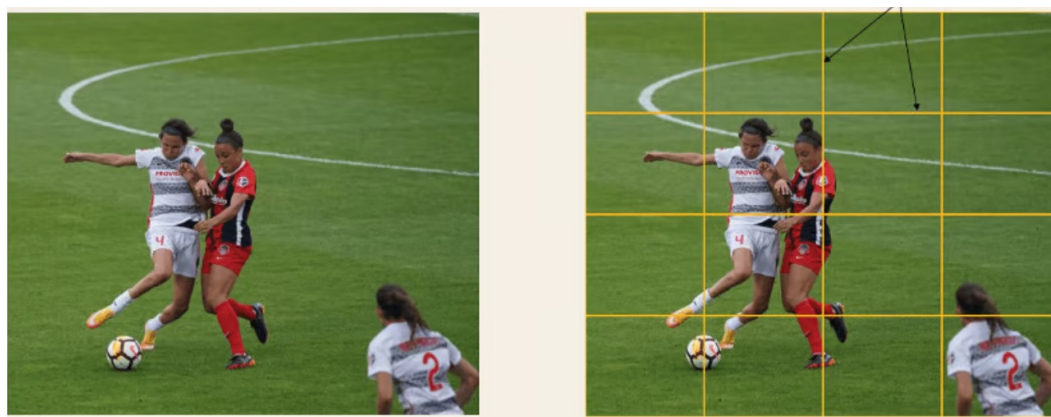


Figure 5: Descripción de la imagen dividida en  $N=4$

### 3.2.2 Regresion de Caja Delimitadora

El algoritmo determina las cajas delimitadoras, que corresponden a rectángulos que resaltan todos los objetos de la imagen. Podemos tener tantas cajas delimitadoras como objetos haya en una imagen determinada. Esto es especialmente importante durante la fase de entrenamiento del modelo. El algoritmo determina los atributos de estos recuadros delimitadores utilizando un único módulo de regresión con el siguiente formato, donde  $Y$  es la representación vectorial final de cada recuadro delimitador.

$$Y = [pc, bx, by, bh, bw, c1, c2]$$

Donde:

- La variable  $pc$  corresponde a la puntuación de probabilidad de la cuadrícula que contiene un objeto. Por ejemplo en la Figura 6 , se muestra todas las cuadrículas en rojo tendrán una puntuación de probabilidad superior a cero. La imagen de la

derecha es la versión simplificada, ya que la probabilidad de cada celda amarilla es cero (insignificante).

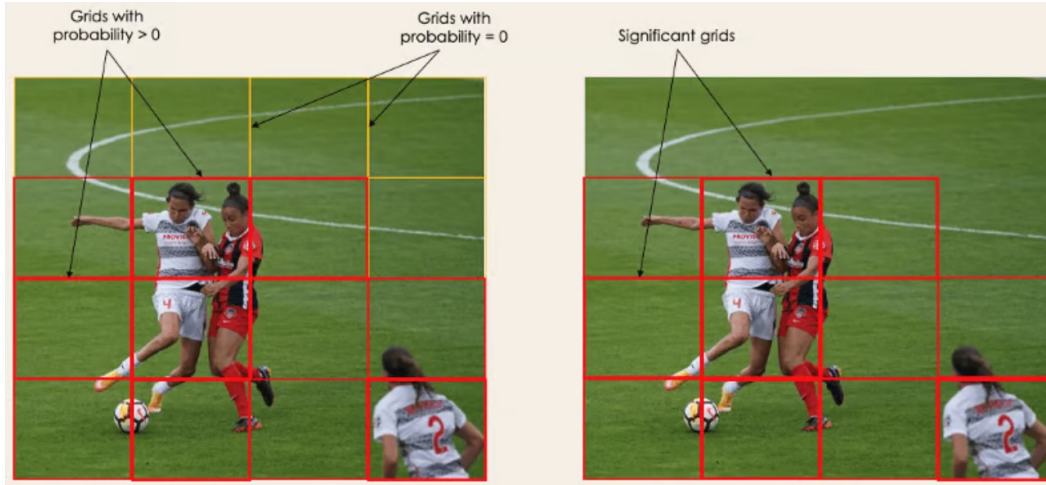


Figure 6: Descripción de puntuación asignada

- Las variables  $bx$ ,  $by$  son las coordenadas  $x$  e  $y$  del centro del cuadro delimitador con respecto a la celda de la malla envolvente.
- Las variables  $bh$ ,  $bw$  corresponden a la altura y la anchura del cuadro delimitador con respecto a la celda de la rejilla envolvente.
- Las variables  $c1$  y  $c2$  corresponden a las dos clases Jugador y Pelota. Podemos tener tantas clases como requiera tu caso de uso.

Así formalmente haremos una descripción de la regresión de caja Delimitadora. Sea una caja delimitadora real

$$B^* = (x^*, y^*, w^*, h^*)$$

y una caja predicha

$$\hat{B} = (\hat{x}, \hat{y}, \hat{w}, \hat{h})$$

donde  $(x, y)$  representan las coordenadas del centro y  $(w, h)$  el ancho y alto, normalizados respecto al tamaño de la imagen.

En Yolo, el modelo produce parámetros intermedios  $(t_x, t_y, t_w, t_h)$  que se transforman

a coordenadas absolutas mediante:

$$\hat{x} = \sigma(t_x) + c_x, \quad (3)$$

$$\hat{y} = \sigma(t_y) + c_y, \quad (4)$$

$$\hat{w} = p_w \cdot e^{t_w}, \quad (5)$$

$$\hat{h} = p_h \cdot e^{t_h}, \quad (6)$$

donde:

- $(c_x, c_y)$  son las coordenadas de la esquina superior izquierda de la celda de la cuadrícula a la que pertenece la predicción.
- $(p_w, p_h)$  son las dimensiones del *anchor box* asociado.
- $\sigma(\cdot)$  es la función sigmoide,  $\sigma(z) = \frac{1}{1+e^{-z}}$ .

El objetivo de la regresión es minimizar una función de pérdida  $\mathcal{L}_{\text{bbox}}$  que mida la discrepancia entre  $\hat{B}$  y  $B^*$ , usando la Intersección sobre Unión (*IoU*).

$$\mathcal{L}_{\text{IoU}} = 1 - \text{IoU}(\hat{B}, B^*) \quad (7)$$

Así la regresión de caja delimitadora en Yolo consiste en aprender una función

$$f_\theta : \mathbb{R}^d \longrightarrow [0, 1]^2 \times \mathbb{R}^2$$

que, dadas las características extraídas de la imagen, produzca parámetros  $(t_x, t_y, t_w, t_h)$  que, tras su transformación, maximicen la IoU con la caja real.

### 3.2.3 Intersección sobre Union (IOU)

También conocido como índice de Jaccard, en la cual la mayoría de las veces, un solo objeto de una imagen puede tener varias cajas cuadrículadas candidatas a la predicción, aunque no todas sean relevantes. El objetivo del IOU (un valor entre 0 y 1) es descartar esas casillas de la cuadrícula para conservar sólo las que sean relevantes. Ésta es la lógica que hay detrás:



- El usuario define su umbral de selección IOU.
- El algoritmo calcula el IOU de cada celda de la cuadrícula, que es el área de Intersección dividida por el Área de Unión (Figura 7).

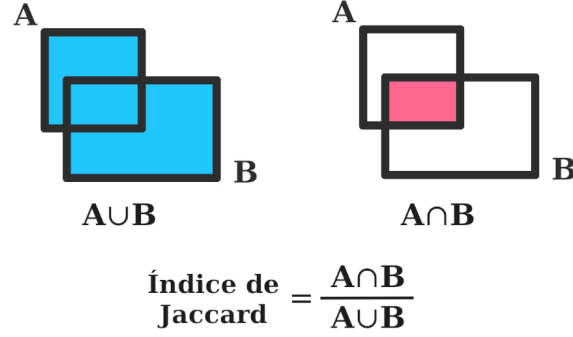


Figure 7: Descripción del calculo del Ind. Jaccard

- El algoritmo ignora la predicción de las celdas de la cuadrícula que tengan un  $\text{IOU} \leq \text{umbral}$  y considera las que tengan un  $\text{IOU} > \text{umbral}$ .

En la figura 8 se muestra una ilustración de la aplicación del proceso de selección de cuadrícula al objeto inferior izquierdo. Podemos observar que el objeto tenía originalmente dos cuadrículas candidatas, pero al final sólo se seleccionó la cuadrícula 2.

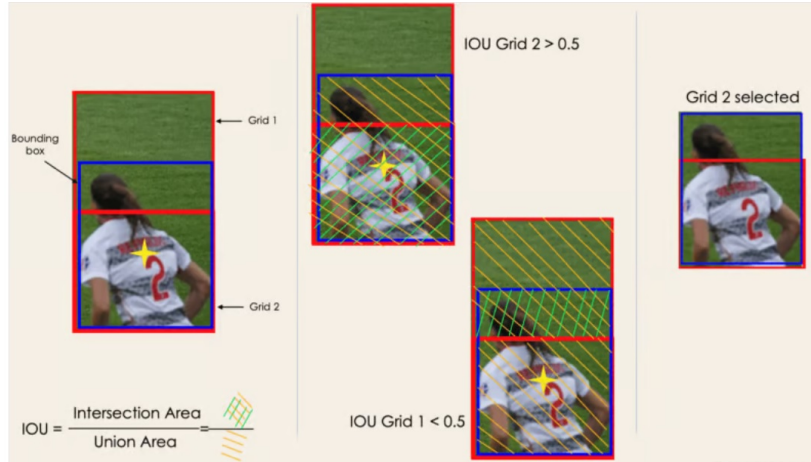


Figure 8: Secuencia de cálculo IOU

Así rigurosamente se definimos  $\text{IoU}$  como:

$$\text{IoU}(B_a, B_b) = \frac{\text{Area}(B_a \cap B_b)}{\text{Area}(B_a) + \text{Area}(B_b) - \text{Area}(B_a \cap B_b)} \quad (8)$$

donde

$$\text{Area}(B) = (x_2 - x_1) \cdot (y_2 - y_1)$$

y la intersección se calcula como:

$$x_{\text{left}} = \max(x_{1a}, x_{1b}), \quad y_{\text{top}} = \max(y_{1a}, y_{1b})$$

$$x_{\text{right}} = \min(x_{2a}, x_{2b}), \quad y_{\text{bottom}} = \min(y_{2a}, y_{2b})$$

Si  $x_{\text{right}} < x_{\text{left}}$  o  $y_{\text{bottom}} < y_{\text{top}}$ , entonces  $\text{Area}(B_a \cap B_b) = 0$ .

### 3.2.4 Non maximum suppression (NMS)

Establecer un umbral para el IOU no siempre es suficiente, porque un objeto puede tener varias casillas con IOU por encima del umbral, y dejar todas esas casillas podría incluir ruido. Aquí es donde podemos utilizar el NMS para conservar sólo las casillas con mayor puntuación de probabilidad de detección.

---

#### Algorithm 1 Supresión No Máxima (NMS)

---

**Require:** Conjunto de cajas  $\mathcal{B} = \{B_1, B_2, \dots, B_n\}$  con coordenadas  $(x_{1i}, y_{1i}, x_{2i}, y_{2i})$ , puntajes  $S(i)$  y umbral  $\tau$ .

**Ensure:** Subconjunto filtrado  $\mathcal{R} \subseteq \mathcal{B}$ .

- 1: Ordenar  $\mathcal{B}$  en orden descendente según  $S(i)$ .
- 2:  $\mathcal{R} \leftarrow \emptyset$
- 3: **while**  $\mathcal{B} \neq \emptyset$  **do**
- 4:      $m \leftarrow \arg \max_{B_i \in \mathcal{B}} S(i)$
- 5:      $\mathcal{R} \leftarrow \mathcal{R} \cup \{B_m\}$
- 6:     Eliminar de  $\mathcal{B}$  todas las cajas  $B_j$  tales que:

$$\text{IoU}(B_m, B_j) > \tau$$

7: **end while**

8: **return**  $\mathcal{R}$

---

## 3.3 Desarrollo de algoritmo

El algoritmo se basa en: la imagen de entrada se divide en una cuadrícula de  $S \times S$  celdas y, para cada celda, Yolo puede detectar  $B$  objetos. Para cada cuadro delimitador (bounding box) detectado, el algoritmo Yolo calcula:

- **Coordenadas y dimensiones:**

$$(x, y, w, h)$$

donde  $(x, y)$  representa la ubicación del cuadro y  $(w, h)$  su ancho y alto.

- **Puntuación de confianza:**

$$p = \text{Pr}(\text{Objeto})$$

que indica la probabilidad de que el cuadro delimitador contenga un objeto.

- **Probabilidades condicionales de clase:**

$$p_i = (\text{Clase}_i \mid \text{Objeto}), \quad i = 1, \dots, K$$

formando una distribución de probabilidad sobre las  $K$  clases posibles.

En consecuencia, para cada cuadro delimitador, el algoritmo Yolo genera un vector de tamaño:

$$R = 4 + 1 + K$$

correspondiente a:

$$(P_r, x, y, w, h, p_1, p_2, \dots, p_K).$$

Durante la etapa de prueba, un objeto de clase  $i$  se detecta si:

$$\text{Pr}(\text{Objeto}) \times \text{Pr}(\text{Clase}_i \mid \text{Objeto}) > \tau$$

donde  $\tau$  es un umbral predefinido.

Dado que en cada celda de la cuadrícula se pueden predecir  $B$  cuadros delimitadores, el tamaño total del vector de salida por celda es:

$$Q = B \times R.$$

## 4 Implementación de YOLO11

En esta sección se presenta la implementación del algoritmo YOLO11 para detección de objetos. La idea central del método YOLO, acrónimo de *You Only Look Once*, consiste en procesar una imagen mediante una red neuronal convolucional profunda y predecir, en una sola pasada, las cajas delimitadoras, las clases y los niveles de confianza asociados a los objetos detectados. A diferencia de los enfoques de dos etapas, YOLO pertenece a la familia de detectores de una etapa, por lo que está orientado a aplicaciones donde se requiere inferencia en tiempo real. El flujo general de YOLO puede resumirse como

imagen de entrada  $\rightarrow$  backbone  $\rightarrow$  neck  $\rightarrow$  head de detección  $\rightarrow$  cajas, clases y confianza.

El *backbone* extrae características visuales de la imagen; el *neck* combina información de diferentes escalas espaciales; y el *head* produce las predicciones finales. Esta estructura permite detectar objetos pequeños, medianos y grandes en una misma imagen.

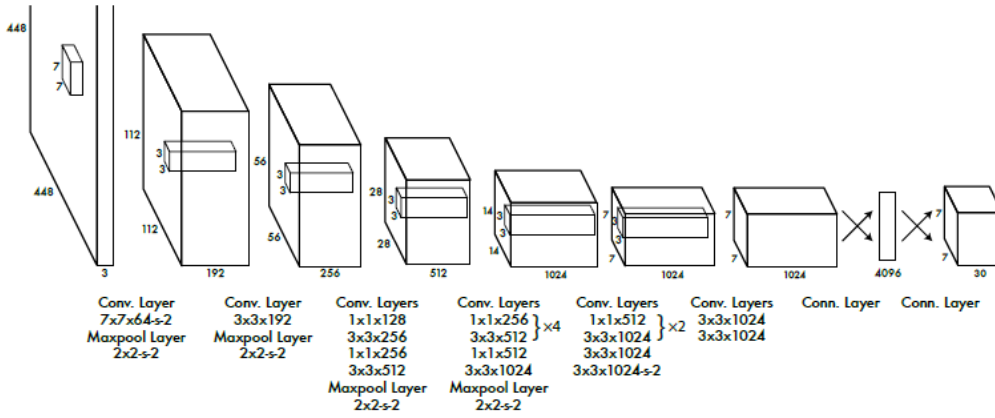


Figure 9: Arquitectura general de YOLO utilizada como referencia conceptual. La imagen de entrada se transforma progresivamente mediante capas convolucionales hasta obtener una representación de salida para detección.

Aunque la arquitectura original de YOLO dividía la imagen en una cuadrícula de tamaño  $S \times S$  y generaba un tensor de salida de la forma

$$S \times S \times (B \cdot 5 + C),$$

donde  $B$  representa el número de cajas por celda y  $C$  el número de clases, las versiones modernas como YOLO11 emplean cabezas de detección más eficientes y predicción multiescala. Sin embargo, la interpretación conceptual se mantiene: el modelo localiza objetos mediante cajas y asigna a cada detección una clase y un nivel de confianza.

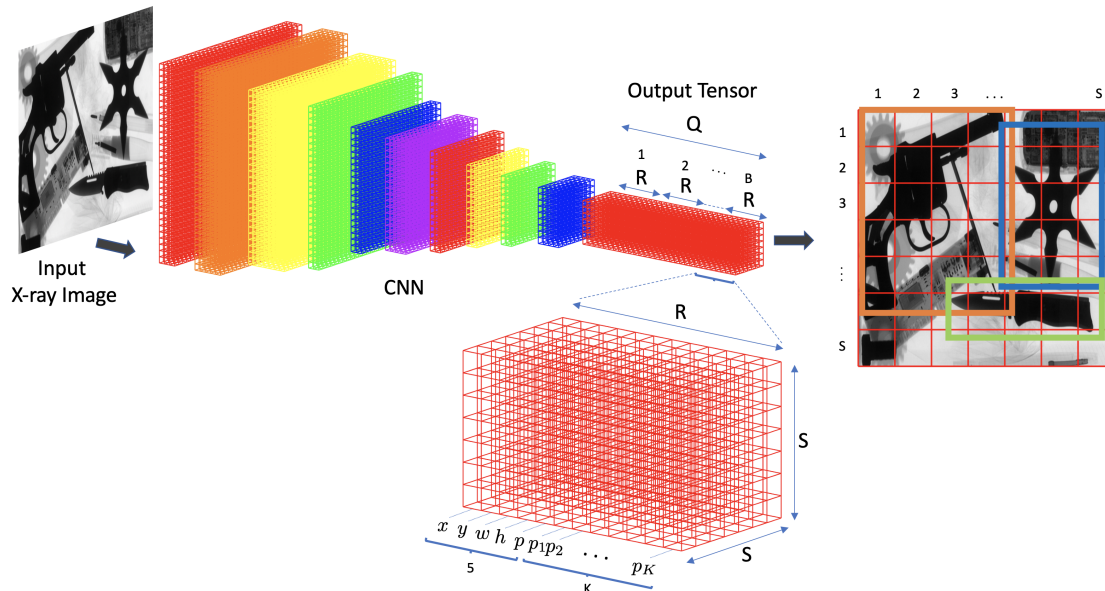


Figure 10: Esquema conceptual de la salida de YOLO: la red transforma la imagen en predicciones estructuradas que contienen posición, tamaño, confianza y clase del objeto.

## 4.1 Modelos preentrenados

Para la implementación se utilizaron modelos preentrenados de la familia YOLO11. Estos pesos permiten realizar detección de objetos sin entrenar desde cero, debido a que ya fueron ajustados previamente sobre conjuntos de datos grandes. En el caso de los modelos de detección de Ultralytics, los pesos se encuentran entrenados sobre COCO, un conjunto de datos ampliamente usado en visión por computadora con 80 clases de objetos.

Los modelos preentrenados se diferencian principalmente por su tamaño, número de parámetros, costo computacional y precisión. La familia de detección de YOLO11 contiene las variantes **n**, **s**, **m**, **l** y **x**. La letra **n** corresponde a *nano*, es decir, el modelo más liviano; mientras que **x** corresponde al modelo más grande y costoso computacionalmente.

Table 1: Modelos YOLO11 preentrenados para detección de objetos sobre COCO.

| Modelo  | Tamaño<br>px | mAP <sub>50-95</sub> | CPU ONNX<br>ms | T4 TensorRT10<br>ms | Parámetros<br>M |
|---------|--------------|----------------------|----------------|---------------------|-----------------|
| YOLO11n | 640          | 39.5                 | 56.1           | 1.5                 | 2.6             |
| YOLO11s | 640          | 47.0                 | 90.0           | 2.5                 | 9.4             |
| YOLO11m | 640          | 51.5                 | 183.2          | 4.7                 | 20.1            |
| YOLO11l | 640          | 53.4                 | 238.6          | 6.2                 | 25.3            |
| YOLO11x | 640          | 54.7                 | 462.8          | 11.3                | 56.9            |

La Tabla 1 muestra que existe un compromiso entre precisión y velocidad. YOLO11n es el modelo más conveniente cuando se prioriza inferencia rápida o uso en equipos con recursos limitados. En cambio, YOLO11m, YOLO11l y YOLO11x son alternativas más precisas, pero requieren mayor capacidad de cómputo.

En el presente trabajo se empleó principalmente el modelo

`yolo11n.pt`,

debido a que permite realizar detección en tiempo real con bajo costo computacional. Esto es especialmente útil cuando se trabaja con cámara web, múltiples cámaras o dispositivos de borde.

## 4.2 Preparación de dataset

Para evaluar un modelo YOLO se requiere organizar el conjunto de imágenes de acuerdo con la estructura esperada por Ultralytics. Si el objetivo es solamente ejecutar inferencia, basta con disponer de imágenes o video. Sin embargo, si se desea calcular métricas como precisión, *recall* o mAP, entonces es necesario contar con anotaciones de verdad terreno.

La estructura recomendada del conjunto de datos es la siguiente:

```
dataset/
|-- images/
|   |-- train/
|   |-- val/
|   `-- test/
```

```
|-- labels/
|   |-- train/
|   |-- val/
|   `-- test/
`-- data.yaml
```

Cada imagen debe tener un archivo de etiqueta con el mismo nombre base y extensión `.txt`. El formato de una anotación YOLO es

$$\text{class\_id} \quad x_c \quad y_c \quad w \quad h,$$

donde  $x_c$  e  $y_c$  representan el centro de la caja, mientras que  $w$  y  $h$  representan el ancho y alto. Todas estas cantidades se normalizan en el intervalo  $[0, 1]$  usando el ancho y alto de la imagen original:

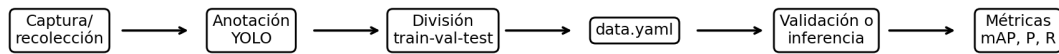
$$x_c = \frac{x_{\text{centro}}}{W}, \quad y_c = \frac{y_{\text{centro}}}{H}, \quad w = \frac{W_{\text{caja}}}{W}, \quad h = \frac{H_{\text{caja}}}{H}.$$

El archivo `data.yaml` define las rutas del conjunto de datos y el nombre de las clases. Un ejemplo mínimo es:

```
path: dataset
train: images/train
val: images/val
test: images/test
```

```
names:
  0: persona
  1: carro
```

El flujo de preparación se resume en la Figura 11. Primero se recolectan las imágenes; luego se anotan las cajas de los objetos; después se divide el conjunto en entrenamiento, validación y prueba; finalmente se ejecuta la validación o inferencia del modelo.



Flujo utilizado para preparar el conjunto de imágenes y evaluar los pesos preentrenados de YOLO11.

Figure 11: Flujo de preparación del dataset para evaluar modelos YOLO11.

### 4.3 Modelos corridos

La ejecución de los modelos se realizó usando la librería `ultralytics`. El procedimiento consiste en cargar los pesos preentrenados, leer una imagen o un flujo de video y aplicar inferencia para obtener las cajas detectadas. En Python, el modelo se carga de la siguiente forma:

```

from ultralytics import YOLO

model = YOLO("yolo11n.pt")
results = model("imagen.jpg")

```

Para una evaluación comparativa entre variantes de YOLO11, se puede ejecutar el siguiente esquema:

```

from ultralytics import YOLO

modelos = [
    "yolo11n.pt",
    "yolo11s.pt",
    "yolo11m.pt",
    "yolo11l.pt",
    "yolo11x.pt"
]

for nombre_modelo in modelos:

```



```

model = YOLO(nombre_modelo)
metrics = model.val(data="data.yaml", imgsz=640)
print(nombre_modelo)
print(metrics.box.map)      # mAP50-95
print(metrics.box.map50)    # mAP50
print(metrics.box.mp)       # precision media
print(metrics.box.mr)       # recall medio

```

En el caso de detección con cámara en tiempo real, se usa una captura de video mediante OpenCV y cada cuadro se envía al modelo. Para cada imagen o cuadro, YOLO devuelve una lista de detecciones con la clase predicha, la confianza y las coordenadas de la caja. Conceptualmente, para cada detección se obtiene

$$(x_1, y_1, x_2, y_2, c, \hat{y}),$$

donde  $(x_1, y_1)$  y  $(x_2, y_2)$  son las esquinas de la caja,  $c$  es el nivel de confianza y  $\hat{y}$  es la clase predicha.

Durante el entrenamiento o ajuste fino, YOLO minimiza una función de pérdida compuesta por tres términos principales:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{dfl}}. \quad (9)$$

El término  $\mathcal{L}_{\text{box}}$  mide el error de localización de la caja;  $\mathcal{L}_{\text{cls}}$  mide el error de clasificación; y  $\mathcal{L}_{\text{dfl}}$  ayuda a refinar la predicción de los bordes de la caja mediante *Distribution Focal Loss*. Esta idea se muestra en la Figura 12.

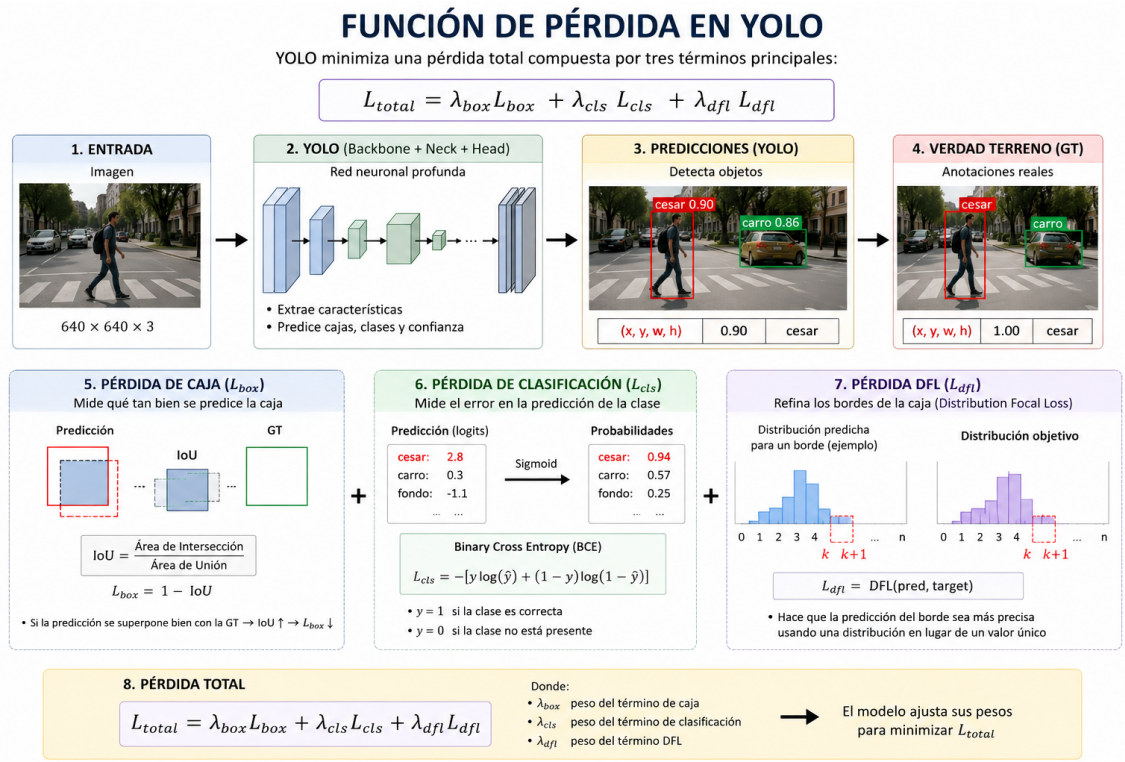


Figure 12: Función de pérdida de YOLO compuesta por localización de caja, clasificación y refinamiento de bordes mediante DFL.

El aprendizaje se realiza por mini-lotes. Para cada mini-lote, el modelo produce predicciones, calcula la pérdida, obtiene gradientes por retropropagación y actualiza sus pesos mediante descenso de gradiente. La actualización general puede escribirse como

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t), \quad (10)$$

donde  $\theta_t$  representa los pesos actuales,  $\eta$  es la tasa de aprendizaje y  $\nabla_{\theta} \mathcal{L}(\theta_t)$  es el gradiente de la función de pérdida respecto de los pesos.

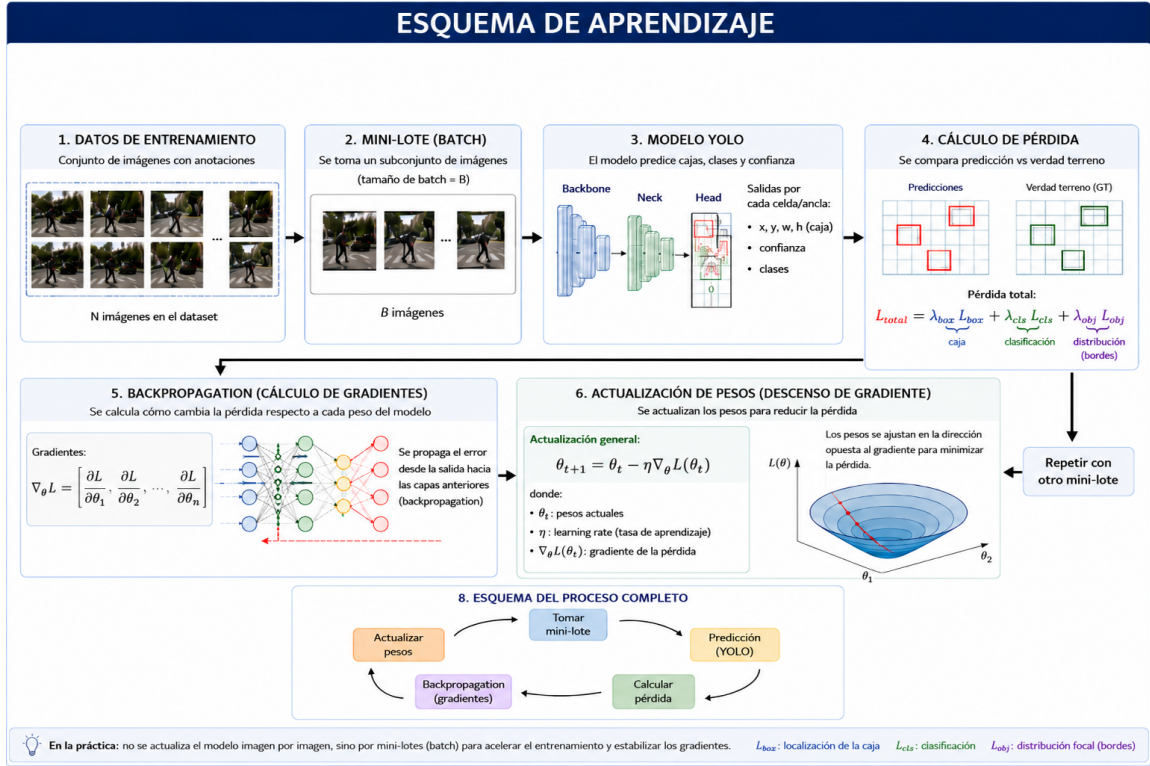


Figure 13: Esquema de aprendizaje de YOLO por mini-lotes: predicción, cálculo de pérdida, retropropagación y actualización de pesos.

## 4.4 Resultados de métricas

Para evaluar un detector de objetos se usan métricas basadas en verdaderos positivos, falsos positivos y falsos negativos. Sea  $TP$  el número de detecciones correctas,  $FP$  el número de detecciones incorrectas y  $FN$  el número de objetos reales no detectados. La precisión y el *recall* se definen como

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (11)$$

La métrica AP, *Average Precision*, corresponde al área bajo la curva precisión–*recall*. En detección de objetos se reporta comúnmente  $mAP_{50}$  y  $mAP_{50-95}$ . La primera considera una detección correcta cuando el IoU entre la caja predicha y la caja real es al menos 0.50. La segunda promedia varios umbrales de IoU desde 0.50 hasta 0.95, por lo que es una métrica más estricta.

Las Figuras 14, 15 y 16 muestran el comportamiento comparativo de los modelos YOLO11 preentrenados. Se observa que al aumentar el tamaño del modelo aumenta

la precisión, pero también crecen la latencia, los parámetros y los FLOPs.

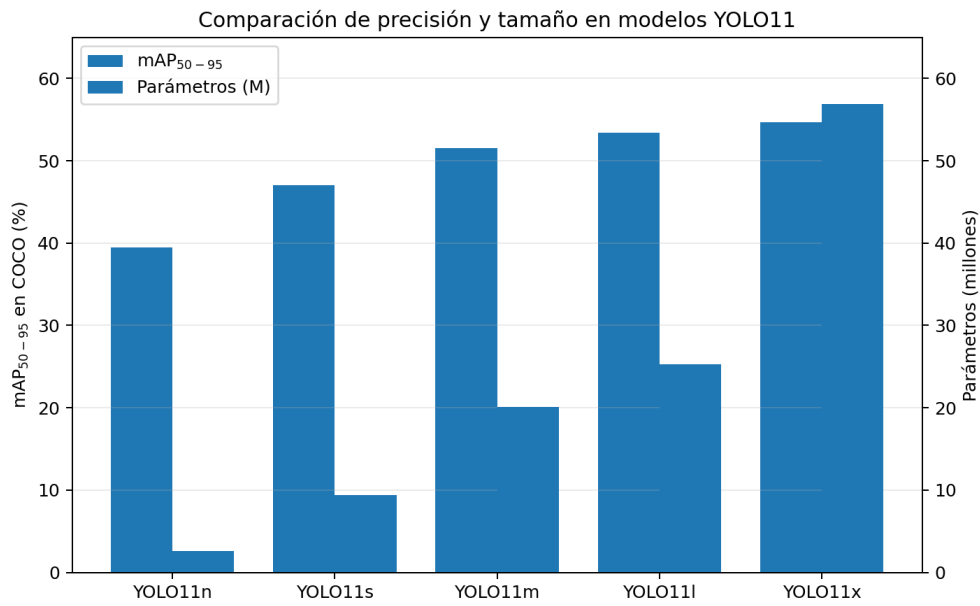


Figure 14: Comparación entre precisión  $mAP_{50-95}$  y número de parámetros de los modelos YOLO11.

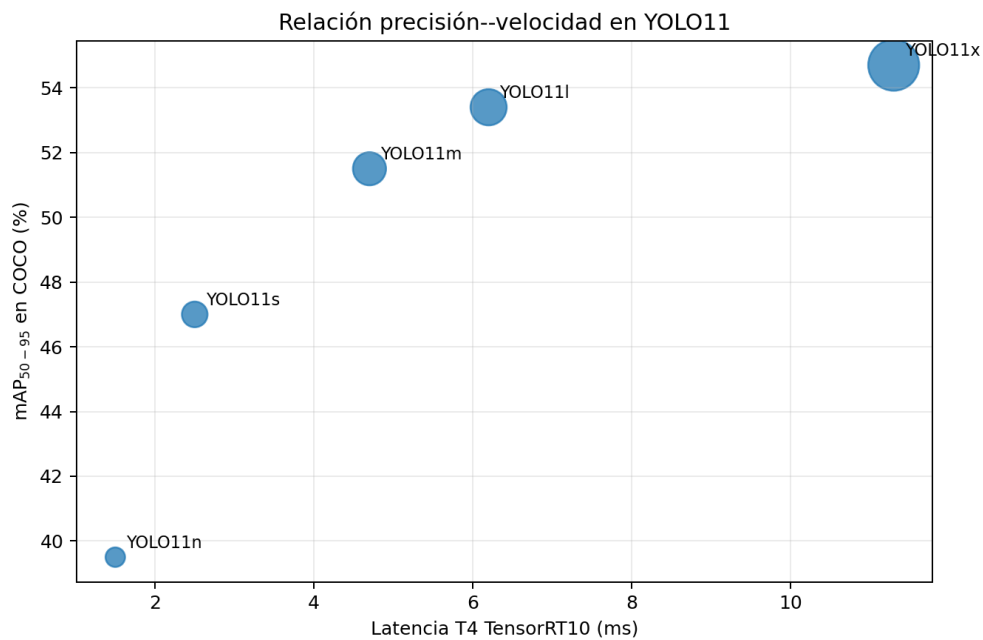


Figure 15: Relación entre precisión y latencia de inferencia en T4 TensorRT10. Los modelos grandes mejoran la precisión, pero aumentan el tiempo de inferencia.

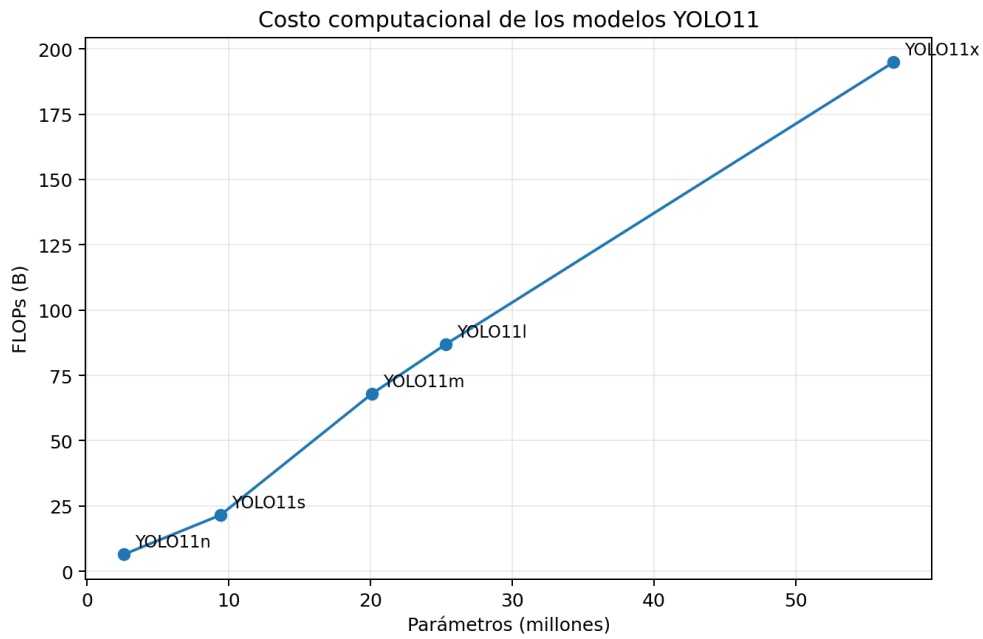


Figure 16: Relación entre parámetros y FLOPs para las variantes YOLO11.

A partir de estos resultados se concluye que YOLO11n es adecuado cuando se busca baja latencia y despliegue en tiempo real. Por otro lado, YOLO11m, YOLO11l y YOLO11x son más convenientes cuando se dispone de GPU y se prioriza la precisión sobre la velocidad. En el contexto de este informe, donde el objetivo es probar un modelo preentrenado y aplicarlo a detección práctica, YOLO11n constituye una elección apropiada por su equilibrio entre velocidad, tamaño y facilidad de ejecución.

## 5 Conclusiones

- Las redes neuronales convolucionales que aprenden representaciones jerárquicas con buen compromiso entre capacidad y parámetros, pero tiene sus limitaciones, porque requiere de datos limpios, sensibilidad al cambio de dominio y requiere muchos calculos numericos.
- Para detección de objetos en tiempo real, el algortimo Yolo es generalmente mejor elección que usar una CNN porque integra en una sola arquitectura.
- Yolo es un detector de una sola etapa que estima simultáneamente cajas y probabilidades por clase. Presenta una excelente relación precisión-latencia frente a detectores de dos etapas.
- Sus principales limitaciones aparecen en objetos muy pequeños/densos y en escenarios con solapamientos fuertes.
- En la práctica, suele ser la opción preferida cuando se busca tiempo real con precisión alta.
- Al usar el algortimo se debe priorizar calidad de datos, calibrar umbrales (conf/IoU) y tamaño de imagen en Yolo.

## 6 Bibliografia

1. O'shea, K., & Nash, R. (2015). An introduction to convolutional neural networks. arXiv preprint arXiv:1511.08458.
2. Calandín Vega, A. (2022). Redes neuronales convolucionales con imágenes hiperspectrales para control de calidad (Doctoral dissertation, Universitat Politècnica de València).
3. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition.
4. ultralyticsdocs Ultralytics Docs: <https://docs.ultralytics.com/>
5. coco Lin, T.-Y. et al. *Microsoft COCO: Common Objects in Context* (2014).